SRH Hochschule Heidelberg

Habit Tracker

Python for Data Science

Group Project Brief

Due Sunday 29.11.2026, 23:59 CET

1. Project overview

Group of 5 students builds a Habit Tracker that runs in the terminal. A user can add habits, mark a habit as done for a day, see streaks and statistics, see a report with simple charts, and save and reload everything from a file. Each student builds one part (one role) on their own. In Week 7 the five parts are joined into one working application.

The examination has two goals: (1) practicing the Python skills taught throughout the teaching weeks (data structures, functions, file handling and error handling in every role; pandas/NumPy in Role C, and matplotlib charts in Role D), and (2) practicing real software-team skills such as agreeing an interface contract early and integrating independently built code without rewriting.

Each student is graded mainly on their own role (70 points: module, tests, team task, peer review, individual note and oral Q&A; see Section 5.2), plus a shared group score for the working demo (20 points) and a challenge for implementing software design patterns (10 points). The group submits their code on GitHub by **Sunday 29 November 2026, 23:59 CET,** and *each student individually defends their own module in a live Q&A during the examination* ***on 30 November / 2 December 2026***.

You are allowed to use AI tools to learn new topics and help you fix logical or syntax errors. In your individual notes, please mention which AI tools you used and how you used them. During the Q&A, you should be able to explain every line of your own code. If you cannot explain your code or give incorrect answers, it may affect your Q&A grade. In serious cases, it may also be considered an academic integrity violation.

2. Interface Contract & Global Conventions

Topic	Key points
Technology	Python 3.10+, run in a terminal on your own computer. Standard library by default; matplotlib only for Role D, pandas/numpy only for Role C. Listed in requirements.txt; later everybody installs it with pip install-r requirements.txt. Tests use plain assert (no pytest).
How the parts fit	One shared data dictionary (habits and log; ids are text, dates are YYYY-MM-DD). main.py (Role E) loads it through storage.py (Role A), changes it with habits.py (Role B), reads numbers from analytics.py (Role C), builds text with reports.py (Role D), and saves it again.
Working method	• **End of week 3:** Everybody writes a stub (placeholder with right function names and fixed return values).

	• **Weeks 4 to 6**: build the real code and run python check_contract.py <module> full. • **Beginning of Week 7**: swap stubs for real files, test together, freeze.
Key rules	Use the exact function names. The contract is locked after Week 3 (changes only add, never rename or remove, and need all five students). Edit only your own files.
Team work	Integration, testing, and review are shared by all five students: everyone writes an integration test, takes small team tasks, and reviews a teammate's work.
Challenge	Required for every group: two design patterns, each implemented by the group inside one student's relevant own file, each with a short rationale in design_patterns.md.

3. The five roles (simplified version)

Every role has one main file, its own unit tests, one integration test file (at least 2 tests), a few team tasks, a review task (see the handout notebook), and a short individual note. More details about integration tests are listed in Section 4.4 of the handout.

Role	File(s)	Functions and Topics	Team Task
A	`storage.py`	load_data, save_data, export_csv Files, JSON, CSV, try/except. A broken file must never crash into the app.	contract.md (Week 3), .gitignore, .gitattributes, requirements.txt; leads the Week 5 mini-integration
B	`habits.py`	add_habit, remove_habit, list_habits, mark_done Dictionaries, lists, input checking, date validation. No date has been added twice.	End of Week 3: copies progress_template.md to progress.md. Creates the design_patterns.md skeleton. Keeps the group section of progress.md.
C	`analytics.py`	current_streak, longest_streak, completion_rate, weekly_summary Loops, dates, sorting, pandas/NumPy. Handles unsorted logs and gaps.	Runs python check_contract.py all full in Week 7 and posts the output
D	`reports.py`	text_chart, save_chart_image, build_report, save_report String formatting, loops, visualization. Clear report even with 10+ habits or none.	README.md (skeleton, install, run, example session)
E	`main.py`, `utils.py`	Menu with options 1 to 9 and 0, at least 3 input helpers (ask_int, ask_date, ask_yes_no, ...), data-file argument The app never crashes on bad input.	None beyond the app

4. Timeline

When	What happens	Hand in
Second half of Week 2- End of Week 3	Form groups and choose roles.	Group list for the instructor
Seconf half of Week 3	Role A fills in contract.md; everybody signs their own block. Role A adds .gitignore and .gitattributes. Role B copies progress_template.md to progress.md and creates the design_patterns.md skeleton (two empty pattern sections). Role D creates the README.md skeleton. Roles A to D push a stub that passes the quick check; Role E pushes a menu stub. The contract is now locked.	Signed contract.md, stubs in the repository
Weeks 4 to 6	Build your module, write your tests, update your own section of progress.md once a week.	Commits
Week 5	Mini-integration: Role A leads, Role E starts python main.py with the real modules that exist and stubs for the rest. Everybody runs their integration test. Not graded, but the note is teamwork evidence.	2 or 3 lines in the group section of progress.md (written by Role B)
Week 6	Peer review: A reviews C, C reviews A, B reviews D, D reviews B, E reviews the integration tests. Roles A to D crash-test main.py and report to Role E. Goal: every module passes full.	Review files in reviews/
First half of Week 7	Integration, ***six steps***: (1) everyone git pull; (2) Role C runs all full, owners fix failures; (3) everyone runs their integration test; (4) everyone goes through the deadlines and checks the deliverables of each milestone; (5) fix cross-module problems (owner fixes); (6) freeze with one check per student, rehearse the demo.	Work in the repository
Week 7	Submission, Sunday 29.11.2026, 23:59 CET	Final repository
Week 8	Demo and oral exam, 30.11 and 02.12.2026 *(Timeslots will be confirmed later for each group)*	On-site Examination

5. Detailed Marking Criteria

5.1 Overall structure (100 points per student)

Component	Points	Scope
Group Demo	**20**	Same score for every student in the group
Individual Role	**70**	Different per student: own module, own unit and integration tests, team task, peer review, individual note, and the oral Q&A. **Each student must be able to explain their own module in the Q&A.**
Challenge Design Patterns	**10**	Same score for every student in the group

5.2 Individual Role (70 points) same scale for all 5 roles

"Required deliverables" means every item of the Individual Checklist in Section 6.3.

Score	Description
65-70	All required deliverables are present and correct. Validation is robust and comprehensive. The solution runs independently without errors. All function signatures match the interface contract exactly. The code is clean, readable, and compliant with PEP 8. The student answers almost all questions in the final examination correctly. During the Q&A session, the student demonstrates an excellent understanding of the implementation, clearly explains and justifies technical decisions, and responds accurately and confidently to the questions.
56-64	All required deliverables are present, or at most one is missing. The solution works well, with at most one minor edge case or a small deviation from the interface contract. The student answers most examination questions correctly. During the Q&A session, the student demonstrates a good understanding of the implementation and provides generally correct explanations, with minor inaccuracies or occasional difficulties in justifying technical decisions.
42-55	The core functionality is implemented but remains incomplete. This may include two or more missing deliverables, inadequate validation, or code that is poorly structured or insufficiently documented. The student answers some examination questions correctly. During the Q&A session, the student demonstrates a partial understanding of the implementation but struggles to explain certain components, justify technical decisions, or respond accurately to technical questions.
41 or less	The solution is broken or incomplete, fails to run independently, or does not comply with the interface contract. The student answers few or none of the examination questions correctly. During the Q&A session, the student demonstrates insufficient understanding of the implementation and is unable to explain the core functionality or provide satisfactory answers to fundamental technical questions.

5.3 Group Demo (20 points, shared)

Criterion	Points
App launches cleanly from one shared entry point, started from a fresh clone of the submitted repository	**4**
Core features from all 5 modules work end-to-end	**6**
Data persists across a restart	**4**
Integrated app handles bad input gracefully	**3**
Group evidence of testing and teamwork (all five integration tests pass, progress.md complete, all five reviews pushed)	**3**

If a student delivers no real module, the group keeps that student's stub; the group demo score falls, and the individual marks of the other four students are not affected.

5.4 Challenge: Software Design Patterns (10 points shared)

Required for every group. Implement 2 of: Singleton, Factory Method, Strategy, Observer, plus a short rationale write-up. A pattern counts as implemented when it is a class in one student's own file, a contract function uses it, and python check_contract.py all full still passes. 4 points per implemented pattern and 1 point for the write-up per pattern. None of the seven teaching weeks cover object-oriented programming or these design patterns; treat this task as independent research, not expected knowledge from the course.

6. Submission Requirements

Your group hands in one private GitHub repository. The repository is marked as it is at the deadline, so freeze the code first (Week 7, step 6 of the integration guide in the handout).

6.1 Deadline and Hand-in

Item	Requirement
Deadline	**Sunday 29 November 2026, 23:59 CET (Week 7). One submission per group. Only commits made before the deadline count.**
Where	A **private** GitHub repository. Add the instructor as a collaborator and share the repository link on Moodle. Further details will be announced later.
Run environment	Python 3.10 or newer on Windows, macOS or Linux, in a terminal (no Jupyter or Colab). No internet is needed to run the app.
Libraries	Standard library by default. Extra libraries only: matplotlib (Role D) and pandas / numpy (Role C). Any extra library must be listed in requirements.txt. No other libraries are allowed. Everybody installs them with pip install-r requirements.txt. Tests use plain assert.
Examination	5-minute group demo and a live individual Q&A on 30 November and 2 December 2026 (Specific timeslots for each group will be confirmed later).

6.2 Repository Details

File or folder	Requirement	Who
storage.py, habits.py, analytics.py, reports.py, main.py + utils.py	Real code (no # STUB line left), exact function names and signatures from the contract, a docstring for every function, PEP 8, no crash on bad input or bad files.	Roles A to E
test_<module>.py	At least 5 meaningful unit tests of your own functions. Temporary files made by a unit test start with _test_unit_.	Each student

File or folder	Requirement	Who
test_integration_A.py to test_integration_E.py	At least 2 integration tests per student, using your own temporary file name (for example _test_int_B.json). Every temporary file a test creates must start with _test_int_ (for example _test_int_A.csv). If a test fails only because of a teammate's module, send a bug report; no marks are lost.	Each student
sample_data.py, check_contract.py	The instructor's original files not edited. python check_contract.py all full must pass.	Nobody edits
contract.md	Filled in and signed by all five students. Changes after Week 3 only in the change log.	Role A fills in
progress.md	A weekly entry from Week 3 to Week 7 in every student's own section, plus the group section (check-ins, mini-integration, peer reviews, integration, final check).	Each student; copy and group section: Role B
README.md	Install, run, an example session, and one section per module.	Role D (skeleton)
design_patterns.md	Two design patterns implemented by the whole group. For each: which pattern and where, which problem it solves, what would be harder without it. The checker must still pass.	Group (skeleton: Role B; each pattern section: group work implementation)
notes/note_A.md to note_E.md	One individual note per student (half a page, see Section 6.3).	Each student
reviews/	review_of_C_by_A.md, review_of_A_by_C.md, review_of_D_by_B.md, review_of_B_by_D.md, review_integration_tests_by_E.md.	Each reviewer
requirements.txt, .gitignore, .gitattributes	requirements.txt only if extra libraries are used. .gitignore keeps __pycache__, temporary test files (_test_int_*, _check_*) and personal data out of the repository.	Role A

6.3 Individual Checklist

- ☐ My module file contains real code, uses the exact contract names, has docstrings and follows PEP 8.
- ☐ python check_contract.py <module> full passes (Roles A to D). Role E: python main.py shows the menu and never crashes on bad input.
- ☐ My unit tests (at least 5) and my integration tests (at least 2) are pushed and pass.

- ☐ My team task is done (Role A: contract.md, Git files, requirements.txt, Week 5 lead. Role B: group section of progress.md. Role C: final full check. Role D: README.md. Role E: none beyond the app).
- ☐ My section of progress.md is updated every week.
- ☐ My peer review is written (A reviews C, C reviews A, B reviews D, D reviews B, E reviews the four integration test files), and I answered the review of my own work (Role E: I fixed the bugs from the crash tests of Roles A to D). Roles A to D also sent crash-test reports to Role E.
- ☐ I signed my own block in contract.md.
- ☐ My note notes/note_X.md contains: (1) my role and what I built, (2) what I learned on my own and how, (3) one bug and its fix, (4) the tools I used (books, websites, AI) and for what, (5) my final checker output (Roles A to D) or the menu output of python main.py (Role E), plus the output of my integration test, (6) the review I wrote (who I reviewed) and my review answer ("changed" or "not changed, because ..."; Role E: which crash-test bugs I fixed), and (7) my file name (notes/note_X.md).
- ☐ I can start the app, run my tests, and explain every function of my module in Q&A.

6.4 Group Submission Checklist

- ☐ The repository is private, the instructor is a collaborator, and the link is shared.
- ☐ On a fresh clone, python main.py starts and the menu appears.
- ☐ python check_contract.py all full passes with the original sample_data.py and check_contract.py. The last lines are pasted in progress.md (Role C runs it, Role B pastes it).
- ☐ All five integration tests pass (python test_integration_X.py).
- ☐ No # STUB line is left in a file that contains real code.
- ☐ The demo works from one entry point: features of all five modules, data kept after a restart, bad input handled.
- ☐ contract.md is complete and signed by all five students.
- ☐ progress.md is up to date, including Week 5 mini-integration, the Week 6 peer reviews and Week 7 integration.
- ☐ README.md, design_patterns.md, requirements.txt (if needed), and .gitignore are finished.
- ☐ All five notes/ files and all five reviews/ files are pushed.
- ☐ Every student has commits under their own GitHub name.